# Stats 21 - HW 3

# Quanliang Chen

Homework copyright Miles Chen. Problems have been adapted from the exercises in Think Python 2nd Ed by Allen B. Downey.

The questions have been entered into this document. You will modify the document by entering your code.

Make sure you run the cell so the requested output is visible. Export it as an HTML file and then print to PDF.

**Homework is an opportunity to practice coding and to practice problem solving.**

**Doing exercises is where you will do most of your learning.**

**Using AI or copying someone else's solutions takes away your learning opportunities. It is also academic dishonesty.**

## Reading

- Chapter 10 - Dictionaries
- Chapter 12 - Tuples
- Chapter 13 - Files

Please keep up with the reading. The chapters are short.

### Modified Exercise based on Think Python 2ed. Chap 11, Section 10, Exercise 11.1

https://greenteapress.com/thinkpython2/html/thinkpython2012.html

Write a function that reads the words in `words.txt` and stores them as keys in a dictionary. It doesn't matter what the values are. Then you can use the `in` operator as a fast way to check whether a string is in the dictionary.

In [1]:
```
def make_word_dict():
    word_dict = {}
    f = open("words.txt")

    for line in f:
        word = line.strip()
        word_dict[word] = None

    f.close()
    return word_dict
```

In [2]:
```
word_dict = make_word_dict()
"hello" in word_dict
```

Out[2]:
```
True
```

## Modified Exercise based on Think Python 2ed. Chap 11, Section 10, Exercise 11.4

https://greenteapress.com/thinkpython2/html/thinkpython2012.html

In HW2 Exercise 10.7, you created a function called `has_duplicates()`. It takes a list as a parameter and returns `True` if there is any object that appears more than once in the list.

Use a dictionary to write a faster, simpler version of `has_duplicates()`.

In [3]:
```
def has_duplicates(t):
    d = {}

    for item in t:
        if item in d:
            return True
        d[item] = True

    return False
```

In [4]:
```
has_duplicates(['a','b','b','c'])
```

Out[4]:
```
True
```

In [5]:
```
has_duplicates(['a','b','c','a'])
```

Out[5]:
```
True
```

## Modified Exercise based on Think Python 2ed. Chap 11, Section 10, Exercise 11.5

https://greenteapress.com/thinkpython2/html/thinkpython2012.html

A Caesar cipher is a weak form of encryption that involves 'rotating' each letter by a fixed number of places. To rotate a letter means to shift it through the alphabet, wrapping around to the beginning if necessary. "A" rotated by 3 is "D". "Z" rotated by 1 is "A".

Two words are "rotate pairs" if you can rotate one of them and get the other. For example, "cheer" rotated by 7 is "jolly".

Write a script that reads in the wordlist `words.txt` and finds all the rotate pairs of words that are 5 letters or longer.

One function that could be helpful is the function `ord()` which converts a character to a numeric code. Keep in mind that numeric codes for uppercase and lowercase

letters are different.

Some hints:

- you can write helper functions, such as a function that will rotate a letter by a certain number and/or another function that will rotate a word by a number of letters
- to keep your script running quickly, you should use the wordlist dictionary from exercise 11.1

In [6]:
```
def rotate_word(word, n):
    result = ""

    for letter in word:
        old_code = ord(letter) - ord("a")
        new_code = (old_code + n) % 26
        new_letter = chr(new_code + ord("a"))
        result += new_letter

    return result


for word in word_dict:
    if len(word) >= 5:
        for n in range(1, 26):
            rotated = rotate_word(word, n)

            if rotated in word_dict:
                print(word, "rotated by", n, "is", rotated)
```

```
abjurer rotated by 13 is nowhere
adder rotated by 1 is beefs
ahull rotated by 6 is gnarr
alkyd rotated by 4 is epoch
alula rotated by 19 is tenet
ambit rotated by 18 is setal
anteed rotated by 1 is bouffe
aptly rotated by 19 is timer
arena rotated by 17 is river
baiza rotated by 18 is tsars
banjo rotated by 4 is ferns
beefs rotated by 25 is adder
benni rotated by 16 is ruddy
biffs rotated by 6 is holly
bolls rotated by 6 is hurry
bombyx rotated by 6 is hushed
bouffe rotated by 25 is anteed
bourg rotated by 20 is viola
buffi rotated by 6 is hallo
bulls rotated by 6 is harry
bunny rotated by 17 is sleep
butyl rotated by 6 is hazer
chain rotated by 6 is ingot
cheer rotated by 7 is jolly
clasp rotated by 15 is raphe
cogon rotated by 16 is sewed
commy rotated by 16 is secco
corky rotated by 20 is wiles
craal rotated by 13 is penny
credo rotated by 16 is shute
creel rotated by 13 is perry
cubed rotated by 10 is melon
curly rotated by 20 is wolfs
cushy rotated by 20 is wombs
danio rotated by 4 is herms
dated rotated by 15 is spits
dawted rotated by 15 is splits
dazed rotated by 15 is spots
didos rotated by 12 is pupae
dolls rotated by 19 is wheel
drips rotated by 11 is octad
ebony rotated by 16 is uredo
eches rotated by 6 is kinky
epoch rotated by 22 is alkyd
fadge rotated by 14 is torus
fagot rotated by 14 is touch
ferns rotated by 22 is banjo
fills rotated by 6 is lorry
fizzy rotated by 5 is kneed
frena rotated by 17 is wiver
frere rotated by 13 is serer
fusion rotated by 6 is layout
ganja rotated by 4 is kerne
gassy rotated by 12 is smeek
ginny rotated by 6 is motte
gnarl rotated by 17 is xeric
gnarr rotated by 20 is ahull
golem rotated by 6 is murks
golly rotated by 6 is murre
green rotated by 13 is terra
```

```
gulfs rotated by 6 is marly
gulls rotated by 6 is marry
gummy rotated by 6 is masse
gunny rotated by 6 is matte
hallo rotated by 20 is buffi
harry rotated by 20 is bulls
hazer rotated by 20 is butyl
herms rotated by 22 is danio
hints rotated by 11 is styed
hoggs rotated by 12 is tasse
holly rotated by 20 is biffs
hotel rotated by 7 is ovals
hurry rotated by 20 is bolls
hushed rotated by 20 is bombyx
ingot rotated by 20 is chain
inkier rotated by 7 is purply
jerky rotated by 9 is snath
jiffs rotated by 6 is polly
jimmy rotated by 6 is posse
jinni rotated by 6 is potto
jinns rotated by 6 is potty
johns rotated by 6 is punty
jolly rotated by 19 is cheer
kerne rotated by 22 is ganja
kinky rotated by 20 is eches
kneed rotated by 21 is fizzy
lallan rotated by 4 is pepper
later rotated by 7 is shaly
latex rotated by 7 is shale
layout rotated by 20 is fusion
linum rotated by 6 is rotas
lorry rotated by 20 is fills
luffs rotated by 6 is rally
lutea rotated by 4 is pyxie
manful rotated by 7 is thumbs
marly rotated by 20 is gulfs
marry rotated by 20 is gulls
masse rotated by 20 is gummy
matte rotated by 20 is gunny
melon rotated by 16 is cubed
mills rotated by 6 is sorry
mocha rotated by 6 is suing
molas rotated by 6 is surgy
motte rotated by 20 is ginny
muffs rotated by 6 is sally
mulch rotated by 6 is sarin
mumms rotated by 6 is sassy
munch rotated by 6 is satin
murks rotated by 20 is golem
murre rotated by 20 is golly
muumuu rotated by 10 is weewee
noggs rotated by 6 is tummy
nowhere rotated by 13 is abjurer
nulls rotated by 6 is tarry
nutty rotated by 6 is tazze
octad rotated by 15 is drips
ovals rotated by 19 is hotel
oxter rotated by 7 is vealy
pecan rotated by 4 is tiger
penny rotated by 13 is craal
```

```
pepper rotated by 22 is lallan
perry rotated by 13 is creel
polly rotated by 20 is jiffs
posse rotated by 20 is jimmy
potto rotated by 20 is jinni
potty rotated by 20 is jinns
primero rotated by 3 is sulphur
pulpy rotated by 6 is varve
punty rotated by 20 is johns
pupae rotated by 14 is didos
purply rotated by 19 is inkier
pyxie rotated by 22 is lutea
rally rotated by 20 is luffs
raphe rotated by 11 is clasp
ratan rotated by 4 is vexer
river rotated by 9 is arena
rotas rotated by 20 is linum
ruddy rotated by 10 is benni
sally rotated by 20 is muffs
sarin rotated by 20 is mulch
sassy rotated by 20 is mumms
satin rotated by 20 is munch
secco rotated by 10 is commy
serer rotated by 13 is frere
setal rotated by 8 is ambit
sewed rotated by 10 is cogon
shale rotated by 19 is latex
shaly rotated by 19 is later
sheer rotated by 1 is tiffs
shute rotated by 10 is credo
sleep rotated by 9 is bunny
smeek rotated by 14 is gassy
snath rotated by 17 is jerky
sneer rotated by 1 is toffs
sorry rotated by 20 is mills
spits rotated by 11 is dated
splits rotated by 11 is dawted
spots rotated by 11 is dazed
steeds rotated by 1 is tuffet
steer rotated by 1 is tuffs
styed rotated by 15 is hints
suing rotated by 20 is mocha
sulphur rotated by 23 is primero
surgy rotated by 20 is molas
tarry rotated by 20 is nulls
tasse rotated by 14 is hoggs
tazze rotated by 20 is nutty
teloi rotated by 3 is whorl
tenet rotated by 7 is alula
terra rotated by 13 is green
thumbs rotated by 19 is manful
tiffs rotated by 25 is sheer
tiger rotated by 22 is pecan
timer rotated by 7 is aptly
toffs rotated by 25 is sneer
torus rotated by 12 is fadge
touch rotated by 12 is fagot
tsars rotated by 8 is baiza
tuffet rotated by 25 is steeds
tuffs rotated by 25 is steer
```

```
tummy rotated by 20 is noggs
uredo rotated by 10 is ebony
varve rotated by 20 is pulpy
vealy rotated by 19 is oxter
vexer rotated by 22 is ratan
viola rotated by 6 is bourg
weewee rotated by 16 is muumuu
wheel rotated by 7 is dolls
whorl rotated by 23 is teloi
wiles rotated by 6 is corky
wiver rotated by 9 is frena
wolfs rotated by 6 is curly
wombs rotated by 6 is cushy
xeric rotated by 9 is gnarl
```

## Modified Exercise based on Think Python 2ed. Chap 12, Section 10, Exercise 12.2

https://greenteapress.com/thinkpython2/html/thinkpython2013.html

Write a program that reads a word list from a file (see Section 9.1) and finds all the sets of words that are anagrams.

After finding all anagram sets, print the list of all anagram sets that have 6 or more entries in it.

Here is an example of what the output might look like:

```
['abets', 'baste', 'bates', 'beast', 'beats', 'betas',
'tabes']
['acers', 'acres', 'cares', 'carse', 'escar', 'races',
'scare', 'serac']
['acred', 'arced', 'cadre', 'cared', 'cedar', 'raced']
...
```

Hint:

First traverse the entire wordlist to build a dictionary that maps from a collection of letters to a list of words that can be spelled with those letters. The question is, how can you represent the collection of letters in a way that can be used as a key? i.e. The word "eat" and the word "tea" should be in a list associated with a key ('a','e','t').

In [7]:
```python
def make_anagram_dict():
    anagram_dict = {}

    with open("words.txt") as f:
        for line in f:
            word = line.strip()
            key = tuple(sorted(word))
            anagram_dict.setdefault(key, []).append(word)

    return anagram_dict


for words in make_anagram_dict().values():
```

```
    if len(words) >= 6:
        print(words)
```

```
['abets', 'baste', 'bates', 'beast', 'beats', 'betas', 'tabes']
['acers', 'acres', 'cares', 'carse', 'escar', 'races', 'scare', 'serac']
['acred', 'arced', 'cadre', 'cared', 'cedar', 'raced']
['aiders', 'deairs', 'irades', 'raised', 'redias', 'resaid']
['airts', 'astir', 'sitar', 'stair', 'stria', 'tarsi']
['alert', 'alter', 'artel', 'later', 'ratel', 'taler']
['alerting', 'altering', 'integral', 'relating', 'tanglier', 'triangle']
['alerts', 'alters', 'artels', 'estral', 'laster', 'ratels', 'salter', 'sl
ater', 'staler', 'stelar', 'talers']
['alevin', 'alvine', 'valine', 'veinal', 'venial', 'vineal']
['algins', 'aligns', 'lasing', 'liangs', 'ligans', 'lingas', 'signal']
['aliens', 'alines', 'elains', 'lianes', 'saline', 'silane']
['aligners', 'engrails', 'nargiles', 'realigns', 'signaler', 'slangier']
['amens', 'manes', 'manse', 'means', 'mensa', 'names', 'nemas']
['anestri', 'nastier', 'ratines', 'retains', 'retinas', 'retsina', 'staine
r', 'stearin']
['angriest', 'astringe', 'ganister', 'gantries', 'granites', 'ingrates',
'rangiest']
['apers', 'asper', 'pares', 'parse', 'pears', 'prase', 'presa', 'rapes',
'reaps', 'spare', 'spear']
['ardeb', 'barde', 'bared', 'beard', 'bread', 'debar']
['ardebs', 'bardes', 'beards', 'breads', 'debars', 'sabred', 'serdab']
['ares', 'arse', 'ears', 'eras', 'rase', 'sear', 'sera']
['aretes', 'easter', 'eaters', 'reseat', 'seater', 'teaser']
['aridest', 'astride', 'diaster', 'disrate', 'staider', 'tardies', 'tirade
s']
['aril', 'lair', 'liar', 'lira', 'rail', 'rial']
['ariled', 'derail', 'dialer', 'laired', 'railed', 'redial', 'relaid']
['arils', 'lairs', 'liars', 'liras', 'rails', 'rials']
['arles', 'earls', 'lares', 'laser', 'lears', 'rales', 'reals', 'seral']
['armets', 'master', 'maters', 'matres', 'ramets', 'stream', 'tamers']
['arrest', 'rarest', 'raster', 'raters', 'starer', 'tarres', 'terras']
['artiest', 'artiste', 'attires', 'iratest', 'ratites', 'striate', 'tastie
r']
['ashed', 'deash', 'hades', 'heads', 'sadhe', 'shade']
['aspen', 'napes', 'neaps', 'panes', 'peans', 'sneap', 'spean']
['aspers', 'parses', 'passer', 'prases', 'repass', 'spares', 'sparse', 'sp
ears']
['aspirer', 'parries', 'praiser', 'rapiers', 'raspier', 'repairs']
['ates', 'east', 'eats', 'etas', 'sate', 'seat', 'seta', 'teas']
['bares', 'baser', 'bears', 'braes', 'saber', 'sabre']
['canter', 'centra', 'nectar', 'recant', 'tanrec', 'trance']
['canters', 'nectars', 'recants', 'scanter', 'tanrecs', 'trances']
['capers', 'crapes', 'escarp', 'pacers', 'parsec', 'recaps', 'scrape', 'se
cpar', 'spacer']
['caress', 'carses', 'crases', 'escars', 'scares', 'seracs']
['caret', 'carte', 'cater', 'crate', 'react', 'recta', 'trace']
['carets', 'cartes', 'caster', 'caters', 'crates', 'reacts', 'recast', 'tr
aces']
['carpels', 'clasper', 'parcels', 'placers', 'reclasp', 'scalper']
['claroes', 'coalers', 'escolar', 'oracles', 'recoals', 'solacer']
['corset', 'coster', 'escort', 'rectos', 'scoter', 'sector']
['cruet', 'curet', 'cuter', 'eruct', 'recut', 'truce']
['cruets', 'cruset', 'curets', 'eructs', 'rectus', 'recuts', 'truces']
['dater', 'derat', 'rated', 'tared', 'trade', 'tread']
['dearths', 'hardest', 'hardset', 'hatreds', 'threads', 'trashed']
['deers', 'drees', 'redes', 'reeds', 'seder', 'sered']
['deils', 'delis', 'idles', 'isled', 'sidle', 'slide']
['deist', 'diets', 'dites', 'edits', 'sited', 'stied', 'tides']
['deltas', 'desalt', 'lasted', 'salted', 'slated', 'staled']
```

```
['deposit', 'dopiest', 'podites', 'posited', 'sopited', 'topside']
['diols', 'idols', 'lidos', 'sloid', 'soldi', 'solid']
['drapes', 'padres', 'parsed', 'rasped', 'spader', 'spared', 'spread']
['earings', 'erasing', 'gainers', 'reagins', 'regains', 'reginas', 'searin
g', 'seringa']
['easters', 'reseats', 'searest', 'seaters', 'teasers', 'tessera']
['easting', 'eatings', 'ingates', 'ingesta', 'seating', 'teasing']
['elastin', 'entails', 'nailset', 'salient', 'saltine', 'tenails']
['emits', 'items', 'metis', 'mites', 'smite', 'stime', 'times']
['empires', 'emprise', 'epimers', 'imprese', 'premies', 'premise', 'spirem
e']
['enosis', 'eosins', 'essoin', 'noesis', 'noises', 'ossein', 'sonsie']
['enters', 'nester', 'rentes', 'resent', 'tenser', 'ternes']
['esprit', 'priest', 'ripest', 'sprite', 'stripe', 'tripes']
['esprits', 'persist', 'priests', 'spriest', 'sprites', 'stirpes', 'stripe
s']
['ester', 'reest', 'reset', 'steer', 'stere', 'terse', 'trees']
['esters', 'reests', 'resets', 'serest', 'steers', 'steres']
['estrange', 'grantees', 'greatens', 'negaters', 'reagents', 'sergeant']
['estrin', 'inerts', 'insert', 'inters', 'niters', 'nitres', 'sinter', 'tr
iens', 'trines']
['estrous', 'oestrus', 'ousters', 'sourest', 'souters', 'stoures', 'tussor
e']
['eviler', 'levier', 'liever', 'relive', 'revile', 'veiler']
['hales', 'heals', 'leash', 'selah', 'shale', 'sheal']
['hassel', 'hassle', 'lashes', 'selahs', 'shales', 'sheals']
['hectors', 'rochets', 'rotches', 'tochers', 'torches', 'troches']
['lapse', 'leaps', 'pales', 'peals', 'pleas', 'salep', 'sepal', 'spale']
['lavers', 'ravels', 'salver', 'serval', 'slaver', 'velars', 'versal']
['laves', 'salve', 'slave', 'vales', 'valse', 'veals']
['leapt', 'lepta', 'palet', 'petal', 'plate', 'pleat']
['least', 'setal', 'slate', 'stale', 'steal', 'stela', 'taels', 'tales',
'teals', 'tesla']
['leasts', 'slates', 'stales', 'steals', 'tassel', 'teslas']
['luster', 'lustre', 'result', 'rustle', 'sutler', 'ulster']
['lusters', 'lustres', 'results', 'rustles', 'sutlers', 'ulsters']
['mates', 'meats', 'satem', 'steam', 'tames', 'teams']
['merits', 'mister', 'miters', 'mitres', 'remits', 'smiter', 'timers']
['nestor', 'noters', 'stoner', 'tenors', 'tensor', 'toners', 'trones']
['notes', 'onset', 'seton', 'steno', 'stone', 'tones']
['opts', 'post', 'pots', 'spot', 'stop', 'tops']
['painters', 'pantries', 'pertains', 'pinaster', 'pristane', 'repaints']
['palest', 'palets', 'pastel', 'petals', 'plates', 'pleats', 'septal', 'st
aple']
['palters', 'persalt', 'plaster', 'platers', 'psalter', 'stapler']
['parties', 'pastier', 'piaster', 'piastre', 'pirates', 'traipse']
['parts', 'prats', 'sprat', 'strap', 'tarps', 'traps']
['paste', 'pates', 'peats', 'septa', 'spate', 'tapes', 'tepas']
['paster', 'paters', 'prates', 'repast', 'tapers', 'trapes']
['peers', 'peres', 'perse', 'prees', 'prese', 'speer', 'spree']
['peris', 'piers', 'pries', 'prise', 'ripes', 'speir', 'spier', 'spire']
['petrous', 'posture', 'pouters', 'proteus', 'spouter', 'troupes']
['piles', 'plies', 'slipe', 'speil', 'spiel', 'spile']
['realist', 'retails', 'saltier', 'saltire', 'slatier', 'tailers']
['recepts', 'respect', 'scepter', 'sceptre', 'specter', 'spectre']
['reigns', 'renigs', 'resign', 'sering', 'signer', 'singer']
['reins', 'resin', 'rinse', 'risen', 'serin', 'siren']
['resaw', 'sawer', 'sewar', 'sware', 'swear', 'wares', 'wears']
['staw', 'swat', 'taws', 'twas', 'wast', 'wats']
['stow', 'swot', 'tows', 'twos', 'wost', 'wots']
```

## NumPy Exercises

For the following exercises, be sure to print the desired output. You will not receive credit for problems that do not print the desired output.

Some of these exercises will require the use of functions in numpy that may not have been covered in class. Look up documentation on how to use them. I always recommend checking the official reference at https://numpy.org/doc/stable/reference/index.html

In [8]:
```
import numpy as np
rng = np.random.default_rng(seed = 1)
```

### Exercise NP.1:

Task: Create an array `b` of 10 random integers selected between 0-99

Desired output: `[47 51 75 95  3 14 82 94 24 31]` of course yours might be different

In [9]:
```
b = rng.integers(0, 100, size=10)
print(b)
```

```
[47 51 75 95  3 14 82 94 24 31]
```

### NP.1b:

Task: reverse the elements in `b`. Hint: Try slicing the array, but backwards

In [10]:
```
print(b[::-1])
```

```
[31 24 94 82 14  3 95 75 51 47]
```

### NP.2a:

Task: Create an array `c` of 1000 random values selected from a normal distribution centered at 100 with sd = 15, rounded to 1 decimal place. Print only the first 10 values.

Desired output: `[ 92.1  83.9 113.   65.5 126.2  88.6 104.8  96.3 121.9  69.1]` of course your values may be different

In [11]:
```
c = rng.normal(loc=100, scale=15, size=1000)
c = np.round(c, 1)

print(c[:10])
```

```
[106.7  91.9 108.7 105.5 104.4 100.4 108.2  89.   97.6  92.8]
```

### NP.2b:

Perform a Shapiro-Wilk test to see if the values in c come from a normal distribution. Report the p-value and appropriate conclusion.

Look up `scipy.stats.shapiro` for usage and details.

https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.shapiro.html

In [12]:
```
from scipy.stats import shapiro

stat, p_value = shapiro(c)

print("p:", p_value)

if p_value < 0.05:
    print("Reject H0: the data is not consistent with a normal distributi
else:
    print("Fail to reject H0: the data is consistent with a normal distri
```

```
p: 0.7866973956692646
Fail to reject H0: the data is consistent with a normal distribution.
```

### NP.2c:

Identify and print the values in `c` that are more than 3 standard deviations from the mean of `c`. Report the proportion of values that are more than 3 sd from the mean.

Desired output: `[ 54.1 ... 148.3 ]`

`0.32 of the values are beyond 3 sd from the mean.`

In [13]:
```
mean_c = np.mean(c)
sd_c = np.std(c)

outliers = c[np.abs(c - mean_c) > 3 * sd_c]

print(outliers)

proportion = len(outliers) / len(c)

print(proportion, "of the values are beyond 3 sd from the mean.")
```

```
[146.5  53.8  46.8 156.3]
0.004 of the values are beyond 3 sd from the mean.
```

### NP.3:

Task: Make a 3x3 identity matrix called `I3`

Desired output:

```
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

In [14]:
```
I3 = np.identity(3)
print(I3)
```

```
[[1. 0. 0.]
 [0. 1. 0.]
 [0. 0. 1.]]
```

### NP.4a:

Task: Make a 10x10 array of values 1 to 100. Call it X

Desired output:

```
[[  1   2   3   4   5   6   7   8   9  10]
 [ 11  12  13  14  15  16  17  18  19  20]
 [ 21  22  23  24  25  26  27  28  29  30]
 [ 31  32  33  34  35  36  37  38  39  40]
 [ 41  42  43  44  45  46  47  48  49  50]
 [ 51  52  53  54  55  56  57  58  59  60]
 [ 61  62  63  64  65  66  67  68  69  70]
 [ 71  72  73  74  75  76  77  78  79  80]
 [ 81  82  83  84  85  86  87  88  89  90]
 [ 91  92  93  94  95  96  97  98  99 100]]
```

In [15]:
```
X = np.arange(1, 101).reshape(10, 10)
print(X)
```

```
[[  1   2   3   4   5   6   7   8   9  10]
 [ 11  12  13  14  15  16  17  18  19  20]
 [ 21  22  23  24  25  26  27  28  29  30]
 [ 31  32  33  34  35  36  37  38  39  40]
 [ 41  42  43  44  45  46  47  48  49  50]
 [ 51  52  53  54  55  56  57  58  59  60]
 [ 61  62  63  64  65  66  67  68  69  70]
 [ 71  72  73  74  75  76  77  78  79  80]
 [ 81  82  83  84  85  86  87  88  89  90]
 [ 91  92  93  94  95  96  97  98  99 100]]
```

### NP.4b:

Task: Make a copy of X, call it Y (1 line). Replace all values in Y that are not squares with 0 (1 line). see `numpy.isin()`

Desired output:

```
[[  1   0   0   4   0   0   0   0   9   0]
 [  0   0   0   0   0  16   0   0   0   0]
 [  0   0   0   0  25   0   0   0   0   0]
...
 [  0   0   0   0   0   0   0   0   0 100]]
```

In [16]:
```
Y = X.copy()
Y[~np.isin(Y, np.arange(1, 11) ** 2)] = 0

print(Y)
```

```
[[  1   0   0   4   0   0   0   0   9   0]
 [  0   0   0   0   0  16   0   0   0   0]
 [  0   0   0   0  25   0   0   0   0   0]
 [  0   0   0   0   0  36   0   0   0   0]
 [  0   0   0   0   0   0   0   0  49   0]
 [  0   0   0   0   0   0   0   0   0   0]
 [  0   0   0  64   0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0   0]
 [ 81   0   0   0   0   0   0   0   0   0]
 [  0   0   0   0   0   0   0   0   0 100]]
```

### NP.5:

Task: Use `np.tile()` to tile a 2x2 diagonal matrix of integers to make a checkerboard pattern. Call the matrix `checkers`

Desired output:

```
[[1 0 1 0 1 0 1 0]
 [0 1 0 1 0 1 0 1]
 ...
 [1 0 1 0 1 0 1 0]
 [0 1 0 1 0 1 0 1]]
```

In [17]:
```
checkers = np.tile(np.identity(2, dtype=int), (4, 4))

print(checkers)
```

```
[[1 0 1 0 1 0 1 0]
 [0 1 0 1 0 1 0 1]
 [1 0 1 0 1 0 1 0]
 [0 1 0 1 0 1 0 1]
 [1 0 1 0 1 0 1 0]
 [0 1 0 1 0 1 0 1]
 [1 0 1 0 1 0 1 0]
 [0 1 0 1 0 1 0 1]]
```

### NP.6:

Task: convert the values in `f_temp` from Farenheit to celsius. The conversion is subtract 32, then multiply by 5/9. Round to 1 decimal place.

Desired output:

```
[[21.1 21.7 22.2 22.8 23.3 23.9]
 ...
 [34.4 35.  35.6 36.1 36.7 37.2]]
```

In [18]:
```
# do not modify
f_temp  = np.arange(70,100).reshape((5,6))
```

In [19]:
```
c_temp = np.round((f_temp - 32) * 5 / 9, 1)

print(c_temp)
```

```
[[21.1 21.7 22.2 22.8 23.3 23.9]
 [24.4 25.  25.6 26.1 26.7 27.2]
 [27.8 28.3 28.9 29.4 30.  30.6]
 [31.1 31.7 32.2 32.8 33.3 33.9]
 [34.4 35.  35.6 36.1 36.7 37.2]]
```

### NP.7:

Task: Convert values in the matrix `x` into z-scores by column, call it matrix `z`. That is: each column should have a mean of 0 and std of 1. (subtract the column mean, and divide by the column std). (not required, but see if you can do this in one line)

Print the column means and column std to show that they have been standardized.

Desired output:

```
[[ 0.41148428  1.04307072 -1.43031951]
 [-0.34069129 -1.14825432 -0.77270135]
...
 [-0.60705751 -1.08536687 -0.57430135]
 [-1.28156585 -0.81250928  1.52877401]]

[-1.16573418e-16  7.77156117e-17  1.44328993e-16]
[1. 1. 1.]
```

In [20]:
```
# do not modify
rng = np.random.default_rng(seed = 100)
x = rng.integers(0, 100, size = 30).reshape(10,3)
```

In [21]:
```
z = (x - np.mean(x, axis=0)) / np.std(x, axis=0)

print(z)
print(np.mean(z, axis=0))
print(np.std(z, axis=0))
```

```
[[ 0.41148428  1.04307072 -1.43031951]
 [-0.34069129 -1.14825432 -0.77270135]
 [-1.00437562 -1.26512499  0.46033272]
 [ 1.34064235  1.36446505  0.50143385]
 [-1.13711249  0.92620005  1.85777132]
 [ 1.07516861 -1.38199565  0.87134407]
 [ 0.19025617 -0.85607765  0.13152363]
 [ 1.38488797  0.60480571 -0.77270135]
 [-1.53532308  0.42949971 -1.34811724]
 [-0.38493691  0.28341137  0.50143385]]
[-1.16573418e-16  7.77156117e-17  1.44328993e-16]
[1. 1. 1.]
```

### NP.8:

Task: Convert values in the matrix `x` into scaled values from 0 to 10. That is take each column and scale values linearly so that the largest value is 10, and the smallest value in the column is 0. Round results to 2 decimal places. Call the result `y`

(Not required, but see if you can do the calculations in one line.)

Desired output:

```
[[0.67 0.88 0.  ]
 [0.41 0.09 0.2 ]
 ...
 [0.   0.66 0.02]
 [0.39 0.61 0.59]]
```

In [22]:
```
# do not modify
rng = np.random.default_rng(seed = 100)
x = rng.integers(0, 100, size = 30).reshape(10,3)
```

In [23]:
```
y = (x - np.min(x, axis=0)) / (np.max(x, axis=0) - np.min(x, axis=0))
y = np.round(y, 2)

print(y)
```

```
[[0.67 0.88 0.  ]
 [0.41 0.09 0.2 ]
 [0.18 0.04 0.57]
 [0.98 1.   0.59]
 [0.14 0.84 1.  ]
 [0.89 0.   0.7 ]
 [0.59 0.19 0.48]
 [1.   0.72 0.2 ]
 [0.   0.66 0.02]
 [0.39 0.61 0.59]]
```

### NP.9:

Task: Replace all NaN values in the matrix `x` with 0.

Desired output:

```
[[76. 83. 12.]
 [59.  8.  0.]
 [44.  0. 58.]
 [ 0. 94. 59.]]
```

In [24]:
```
# do not modify
rng = np.random.default_rng(seed = 100)
x = rng.integers(0, 100, size = 12).reshape(4,3).astype('float')
row = np.array([1, 2, 3])
col = np.array([2, 1, 0])
x[row, col] = np.nan
```

In [25]:
```
x[np.isnan(x)] = 0

print(x)
```

```
[[76. 83. 12.]
 [59.  8.  0.]
 [44.  0. 58.]
 [ 0. 94. 59.]]
```